Программирование в лингвистике

Функции. Рекурсия

Еще две парадигмы программирования

(способы организации кода)

Функциональное программирование	Объектно-ориентированное программирование (ООП)
«Глагольный» подход: мир состоит из списка действий, и субъекты и объекты этих действий не важны.	«Существительный» подход: мир состоит из объектов, у которых есть свои атрибуты и характеристики.
Например, в нашем мире есть функция "приготовить обед", а кто будет в ней участвовать – не важно, хоть котик. Разница будет только в результате.	Например, объект "котик" умеет мурлыкать и мяукать, это его методы. А еще у котика есть уши и хвост – это его атрибуты.

Функции в Python

```
def <имя функции>(*args, **kwargs):
    <тело функции>
    return <возвращаемый объект>
```

Функция – вызываемый объект, содержащий **заранее собранный код**.

Функции часто служат для оптимизации внешнего вида кода. Если нам нужно **повторить одни и те же действия** в разных частях программы, имеет смысл использовать **функции**.

Args (arguments) – это **параметры** функции, аргументы которых необходимо дать в **определенной последовательности**.

Kwargs (keyword arguments) – это **ключевые параметры** функции. Передача аргументов в такие параметры необязательна (рассмотрим это далее).

Не важно, в каком порядке передаются аргументы, если присутствуют **ключи**. Если ключей не присутствует, то последовательность **должна соблюдаться**.

```python
def divide(x, y, rounded=False):
  result = x / y
  if rounded:
    result = round(result)
  return result

print(divide(5, 2, rounded=True))
```

```
2
```

По конвенциям определять функции следует в самом начале кода.

Пример функции

```
def divide(x, y, rounded=False):
  result = x / y
  if rounded:
    result = round(result)
  return result

print(divide(5, 2, rounded=True))
```

```
2
```

Эта функция делит первый аргумент (x) на второй (y). Если ключевой аргумент при вызове функции устанавливается и является True, то результат деления округляется.

> def – элемент синтаксиса Python, говорящий ему, что мы создаем функцию.
> divide – наше название функции.
> x, y – args. Здесь это будут числа, и их важно дать в нужном порядке, чтобы наша функция поняла, где делимое, а где делитель.
> rounded=False – kwarg. Ключевой параметр. При передаче аргумент может отсутствовать. Здесь значение rounded по дефолту является False.
> все, что находится внутри функции (с отступом) – ее тело.
> return – элемент синтаксиса Python, предполагающий, что то, что идет за ним через пробел будет результатом выполнения функции (возвращением). Может отсутствовать.
> divide(5, 2, rounded=True) – вызов функции. Здесь x это 5, y это 2, а ключевой аргумент rounded является True.

Параметры args и kwargs

Args – обязательные параметры функции. У них нет дефолтного значения, поэтому они требуют передачи аргумента.
Здесь это number1.

Kwargs – необязательные параметры функции. При создании функции дефолтные значения передаются через знак присвоения.
Здесь это number2, по дефолту равный 4.

```
def print_numbers(number1, number2=4):
  print(number1, number2)

print_numbers(10, 5)
print_numbers(number2=5, number1=10)
print_numbers(10)
```

```
10 5
10 5
10 4
```

Первый раз мы вызываем эту функцию, передав туда оба числа. Number2 оказывается равным 5.
Во второй раз мы делаем то же самое, только дав аргументы вразнобой, указав их ключи.
В третий раз мы передаем в эту функцию только число 10. Number2 остается равным 4.

Аннотирование типов

```
def repeat(s: str, n: int=5) -> str:
  return s * n

print(repeat("a"))
```

```
aaaaa
```

Можно сделать описание функции **более подробным**. IDE будет давать Вам **подсказки**, основываясь на ее **описании**.

Здесь IDE подскажет Вам, что параметр **s** должен получить **строку**, а параметр **n** – **целое число**. А также напомнит, что **функция** должна возвратить **строку**.

Однако, никакой **ошибки не возникнет**, если что-то из этого пойдет не так.

Еще есть оператор **pass**, который ничего не выполняет. Может использоваться для временного заполнения тела функции.

```
def ill_think_later():
  pass
```

Глобальные и локальные переменные

```
x = 5
def create_a():
  a = "a"

print(x)
print(a)
```

```
5

NameError
```

Неймспейс – пространство имен. Это как будто словарь из нашего кода, в котором содержатся все наши объекты под уникальными названиями.

Так, здесь есть следующие элементы неймспейса:
create_a – функция
a – локальная переменная функции
x – глобальная переменная

Глобальные переменные создаются вне функций или классов. Они доступны практически из любого уголка программы, но лишь после их объявления.

Локальные переменные создаются внутри класса или функции, и могут использоваться только на том же уровне вложения.

Здесь попытка напечатать локальную переменную вне функции приводит к ошибке.

Глобальные и локальные переменные

```
def print_1(x):
  x = 1
  print(x)

x = 5
print_1(x)
print('Global x is', x)
```

```
1
Global x is 5
```

Здесь **глобальная** и **локальная** переменные имеют **одно имя**.
Однако, переменная **x** все же равна **5**, даже после того, как в функции мы **изменили** ее значение на **1**. Как так?

Во время работы функции, когда мы **переопределяем** переменную **x**, переменная в глобальном неймспейсе становится **недоступной**. Но когда функция завершает свою работу, **локальная** переменная **x** **перестает существовать**, и на сцену опять выходит **глобальная переменная** со значением **5**.

Работа с глобальными переменными в функции

- Иногда внутри функции нужно что-то сделать с глобальной переменной.
- Чтобы это работало, нужно объявить переменную в начале функции с помощью ключевого слова global.
- Чрезмерное использование global считается плохой практикой из-за усложнения отладки.
- Лучше передавать переменную как параметр функции и возвращать ее.

```
total = 10

def update_total():
    global total

    total = 20

update_total()
print(total)
```

```
20
```

Лямбда-функции

Вычисление кубов с помощью **def**

```
def microfunc(x):
    return x ** 3
```

Вычисление кубов с помощью **lambda**

```
microfunc = lambda x: x ** 3
```

Когда нам нужно написать **функцию-крошку**, нам необязательно прибегать к классическому синтаксису описания функций.

Вместо этого мы можем создать **лямбда-функцию**, которая позволит записать нужную нам операцию в одну строку.

Лямбда-функции

```
microfunc = lambda x: x ** 3
```

Интерпретация

Что сделать с **аргументом**: возвести **аргумент** в третью степень.

> **microfunc** – название нашей лямбда-функции. Обычно лямбда-функции используются внутри какого-то метода или функции, поэтому объявлять их отдельно необязательно.

> **lambda** – элемент синтаксиса Python, сообщающий ему, что мы начинаем лямбда-функцию.

> **x** – переменная лямбда-функции. Далее с помощью нее мы скажем Python, что делать с аргументом функции.

> **x ** 3** – тело лямбда-функции. То, что мы будем делать с аргументом, переданным в microfunc.

Лямбда-функции

```
words = ["orb", "era", "inc"]
words.sort(key=lambda x: x[-1])
print(*words)
```

```
era orb inc
```

Особенно **лямбда-функции** полезны в качестве **ключей для сортировки**.

Здесь **x[-1]** означает, что каждый элемент будет возвращать **последнюю букву**, по которой и будет сортироваться список слов.

Лямбда-функции от нескольких аргументов

- У лямбда-функции может быть сколько угодно аргументов.
- Аргументы могут быть любых типов, в том числе сами быть функциями.
- Но чаще лямбда-функции используются как аргументы других функций.
- В лямбда-функциях могут быть условия и генераторы.

```
maxx = lambda a, b: a if a > b else b
add = lambda a, b: a + b

def operation_ex(operation, x, y):
    return operation(x, y)

maxx_result = operation_ex(maxx, 3, 5)
# Вернет 5

add_result = operation_ex(add, 3, 5)
# Вернет 8
```

Спасибо за внимание!